



HEITOR RAMOS VIEIRA
GABRIEL COELHO COSTA
OTÁVIO SBAMPATO ANDRADE

RELATÓRIO TRABALHO PRÁTICO COMPILADORES:

ETAPA 2

1ª edição

LAVRAS – MG

2026

**HEITOR RAMOS VIEIRA
GABRIEL COELHO COSTA
OTÁVIO SBAMPATO ANDRADE**

RELATÓRIO TRABALHO PRÁTICO COMPILADORES:

ETAPA 2

1^a edição

Relatório apresentado para a segunda etapa do trabalho prático da matéria de Compiladores (código GCC130), do curso de Ciência da Computação da Universidade Federal de Lavras - MG.

LAVRAS – MG

2026

SUMÁRIO

1	Introdução	2
2	Gramática completa em BNF	3
3	Decisões de Projeto	6
3.1	Estrutura Geral e Símbolo Inicial	6
3.2	Agrupamento e Reestruturação de Tokens	6
3.3	Resolução de Ambiguidades via Diretivas de Precedência	6
3.4	Tratamento do <i>Dangling Else</i>	7
3.5	Uso do Ponto-e-Vírgula como Delimitador	7
3.6	Atribuição como Expressão	7
3.7	Produções Auxiliares Notáveis	8
3.8	Tratamento de Erros	8
4	Dificuldades encontradas	9
4.1	Integração Flex e Bison	9
4.2	Estruturação da Gramática	9
5	FIRST e FOLLOW	10
5.1	Análise dos Conjuntos FIRST e FOLLOW da Gramática	10
5.1.1	Tabela Completa de Conjuntos Sintáticos	10
6	Estados do Autômato LR(0) com conflitos	15
7	Arquivo de Teste e Saída	16
7.1	Código de Entrada (entrada.txt)	16
7.2	Saída Gerada pelo Parser	17
7.3	Tabela de Símbolos Resultante	21
8	Conclusão	22

1 INTRODUÇÃO

Este relatório descreve o desenvolvimento da segunda etapa do compilador para a disciplina GCC130. Nesta segunda fase, o foco é o desenvolvimento de um Analisador Sintático (*parser*), implementado com uso da ferramenta *Bison*. O *parser* é o componente do compilador responsável por analisar a sequência de *tokens* produzida pelo *scanner* e verificar se ela está de acordo com a gramática da linguagem. A partir dessa análise, o parser constrói a **árvore sintática**, que serve de base para as etapas seguintes da compilação, reportando erros sempre que a estrutura do código violar as regras gramaticais definidas.

2 GRAMÁTICA COMPLETA EM BNF

$$\begin{aligned}
 \text{program} &::= \text{global-decl-list} \\
 \text{global-decl-list} &::= \text{global-decl-list global-decl} \\
 &\quad | \quad \epsilon \\
 \text{global-decl} &::= \text{func-decl} \\
 &\quad | \quad \text{var-decl} \\
 \text{func-decl} &::= \mathbf{TYPE ID} (\text{opt-param-list}) \text{ block} \\
 \text{opt-param-list} &::= \text{param-list} \\
 &\quad | \quad \epsilon \\
 \text{param-list} &::= \text{param-list} , \mathbf{TYPE ID} \\
 &\quad | \quad \mathbf{TYPE ID} \\
 \text{func-call} &::= \mathbf{ID} (\text{opt-call-list}) \\
 \text{opt-call-list} &::= \text{func-call-list} \\
 &\quad | \quad \epsilon \\
 \text{func-call-list} &::= \text{func-call-list} , \text{expr} \\
 &\quad | \quad \text{expr} \\
 \text{block} &::= \{ \text{stmt-list} \} \\
 \text{stmt-list} &::= \text{stmt-list stmt} \\
 &\quad | \quad \epsilon \\
 \text{stmt} &::= \text{var-decl} \\
 &\quad | \quad \text{assign-stmt} \\
 &\quad | \quad \text{if-stmt} \\
 &\quad | \quad \text{while-stmt} \\
 &\quad | \quad \text{print-stmt} \\
 &\quad | \quad \text{read-stmt} \\
 &\quad | \quad \text{return-stmt} \\
 &\quad | \quad \text{block}
 \end{aligned}$$

$$\begin{aligned}
\text{var-decl} &::= \mathbf{TYPE} \text{ id-list} ; \\
\text{id-list} &::= \text{id-list} , \text{id-decl} \\
&\quad | \text{id-decl} \\
\text{id-decl} &::= \mathbf{ID} \\
&\quad | \mathbf{ID} = \text{expr} \\
\text{assign-stmt} &::= \text{expr} ; \\
\text{if-stmt} &::= \mathbf{if} (\text{expr}) \text{ block else-clause} \\
\text{else-clause} &::= \varepsilon \\
&\quad | \mathbf{else} \text{ block} \\
&\quad | \mathbf{else} \text{ if-stmt} \\
\text{while-stmt} &::= \mathbf{while} (\text{expr}) \text{ block} \\
\text{print-stmt} &::= \mathbf{print} (\text{print-list}) ; \\
\text{print-list} &::= \text{print-list} , \text{expr} \\
&\quad | \text{expr} \\
\text{read-stmt} &::= \mathbf{read} (\text{read-list}) ; \\
\text{read-list} &::= \text{read-list} , \mathbf{ID} \\
&\quad | \mathbf{ID} \\
\text{return-stmt} &::= \mathbf{return} \text{ opt-expr} ; \\
\text{opt-expr} &::= \text{expr} \\
&\quad | \varepsilon \\
\text{expr} &::= \mathbf{ID} = \text{expr} \\
&\quad | \text{expr} || \text{expr} \\
&\quad | \text{expr} \&\& \text{expr} \\
&\quad | \text{expr} \text{ EQOP } \text{expr} \\
&\quad | \text{expr} \text{ RELOP } \text{expr} \\
&\quad | \text{expr} + \text{expr} \\
&\quad | \text{expr} - \text{expr} \\
&\quad | \text{expr} * \text{expr}
\end{aligned}$$

$$\begin{aligned} & \mid \textit{expr} / \textit{expr} \\ & \mid - \textit{expr} \\ & \mid ! \textit{expr} \\ & \mid \textit{primary-expr} \\ \textit{primary-expr} ::= & \mathbf{ID} \\ & \mid \textit{literal} \\ & \mid (\textit{expr}) \\ & \mid \textit{func-call} \\ \textit{literal} ::= & \textit{INTEGER_LITERAL} \\ & \mid \textit{FLOAT_LITERAL} \\ & \mid \textit{STR_LITERAL} \\ & \mid \textit{BOOL_LITERAL} \end{aligned}$$

3 DECISÕES DE PROJETO

3.1 Estrutura Geral e Símbolo Inicial

O símbolo inicial da gramática é `program`, definido como uma lista (possivelmente vazia) de declarações globais (`global_decl_list`). Essa escolha permite que um programa vazio seja considerado sintaticamente válido, o que simplificou a nossa gramática e é coerente com o comportamento de compiladores modernos.

As declarações globais foram divididas em duas categorias: declarações de função (`func_decl`) e declarações de variável (`var_decl`), ambas permitidas no escopo global.

3.2 Agrupamento e Reestruturação de Tokens

O *lexer* foi adaptado para trabalhar em conjunto com o Bison, exportando os tokens necessários e os campos auxiliares exigidos pela integração entre o Flex e Bison.

Alguns tokens foram agrupados ou adicionados com o objetivo de simplificar a gramática e facilitar a definição de precedência. Entre as principais alterações estão:

- Adição dos tokens `RETURN`, `STR_LITERAL` e `BOOL_LITERAL`;
- Inclusão dos tipos `TYPE_BOOL` e `TYPE_STR` sob o token unificado `TYPE`, de forma que todos os tipos primitivos sejam reconhecidos por uma única categoria sintática;
- Separação do que antes era um único conjunto de operadores relacionais em duas classes distintas: `RELOP` (operadores `<`, `>`, `<=`, `>=`) e `EQOP` (operadores `==` e `!=`). Essa separação é motivada pela necessidade de definir uma precedência entre os operadores.

Símbolos ignoráveis pelo *lexer* como espaços em branco, quebras de linha, comentários de linha e comentários de bloco são consumidos sem gerar tokens, o que tornou o *semicolon* essencial como delimitador explícito de *statements* na nossa linguagem.

3.3 Resolução de Ambiguidades via Diretivas de Precedência

A maior parte das ambiguidades inerentes à gramática de expressões foi resolvida por meio das diretivas de precedência e associatividade do Bison, evitando a necessidade de reescrita da gramática em múltiplos níveis de não-terminais. As diretivas definidas, em ordem crescente de precedência, são:

```
%precedence ASSIGN
```

```
%left OR
```



```
%left AND
%left EQOP
%left RELOP
%left PLUS MINUS
%left MULT DIV
%precedence NOT
%precedence UMINUS
```

Essa hierarquia garante o comportamento esperado para expressões aritméticas, lógicas e relacionais. O operador de atribuição possui a menor precedência e é declarado apenas com `%precedence` (sem associatividade), pois o encadeamento de atribuições é tratado diretamente na regra gramatical de forma recursiva à direita (`ID ASSIGN expr`). Os operadores unários `NOT` e `UMINUS` recebem a maior precedência, com `%prec` aplicado explicitamente nas produções correspondentes.

3.4 Tratamento do *Dangling Else*

O problema do *dangling else* (ambiguidade de linguagens com construções `if/else` opcionais) foi eliminado por uma restrição que escolhemos trazer para a gramática: tanto o corpo de `if` quanto de `else` são **necessariamente** delimitados por chaves (`block`). Essa decisão enrijece um pouquinho a sintaxe, mas não a limita, e remove completamente a ambiguidade sem necessidade de diretivas de precedência adicionais ou reescrita da gramática. Como consequência, construções como `else if` encadeado são suportadas pela produção `else_clause ::= else if_stmt`.

3.5 Uso do Ponto-e-Vírgula como Delimitador

Os *statements* que não encerram naturalmente com um bloco (`var_decl`, `assign_stmt`, `print_stmt`, `read_stmt` e `return_stmt`) utilizam o token `PUNCT_SEMICOLON` como delimitador obrigatório. Isso elimina ambiguidades sobre o fim de um *statement* e reduz conflitos `shift/reduce`, pois o parser não precisa utilizar `lookahead` para determinar onde uma construção termina.

3.6 Atribuição como Expressão

Na gramática, a atribuição foi tratada como um caso de `expr`, e não como um *statement* separado. Isso significa que uma atribuição produz um valor e pode aparecer em qualquer lugar onde uma expressão é esperada. Por exemplo:

```
if (a = 2) { ... }
```

É válido. Nesse caso, `a` recebe 2 e esse mesmo valor é avaliado como condição do `if` (e retorna `true`, claro, entrando no bloco). Esse comportamento é inspirado no C e foi uma escolha deliberada para tornar a linguagem mais flexível.

3.7 Produções Auxiliares Notáveis

Duas produções foram decisões nossas de projeto e merecem destaque pela função que exercem na organização da gramática.

- A produção `literal` agrupa todos os tipos de literais da linguagem (inteiros, ponto flutuante, strings e booleanos) num único não-terminal, evitando repetição nas demais produções.

- A produção `primary_expr` reúne as formas mais simples de expressão: identificadores, literais, chamadas de função e expressões entre parênteses. Ela serve de base para a produção recursiva de `expr`, separando as expressões simples das que envolvem operadores.

3.8 Tratamento de Erros

O tratamento de erros sintáticos foi implementado de forma direta visando facilitar o debug pelo usuário final. Utilizamos a diretiva `%define parse.error verbose` no Bison, substituindo a clássica mensagem genérica de erro sintático por um relatório detalhado de quais tokens eram esperados na posição da falha.

Para aumentar ainda mais a precisão, adaptamos a função `yyerror`. Ela agora utiliza variáveis globais do Lexer como `yylineno`, `column_number` e `yylen` para calcular e informar a linha e a coluna exatas onde o erro ocorreu na leitura do arquivo.

4 DIFICULDADES ENCONTRADAS

Durante o desenvolvimento do analisador sintático, enfrentamos dois desafios práticos principais que exigiram mais tempo de estudo e testes.

4.1 Integração Flex e Bison

Fazer o Bison e o Flex se comunicarem corretamente foi nosso primeiro grande obstáculo. Tivemos dificuldade em inicializar o compartilhamento de dados e o estado entre as duas ferramentas. O rasteio de localização de tokens exigiu ajustes nas configurações com uso da diretiva `extern`, e a tipagem incorreta no início do desenvolvimento chegou a causar falhas de segmentação durante a execução da função `yyparse()`.

4.2 Estruturação da Gramática

O segundo maior desafio foi descobrir como mapear a gramática de forma que ela fosse flexível, sucinta e funcional ao mesmo tempo. No começo, o conjunto de regras cresceu muito rápido e as diversas regras em expressões geraram muito código no gerador. Para contornar e facilitar as coisas, tivemos que refatorar os arquivos para unir regras primitivas redundantes e remover o máximo de recursões inadequadas, mantendo assim o código do Bison limpo e fácil de gerenciar.

5 FIRST E FOLLOW

5.1 Análise dos Conjuntos FIRST e FOLLOW da Gramática

Nesta etapa do relatório, documentamos as computações dos conjuntos *FIRST* e *FOLLOW* para todos os não-terminais da gramática. Para fins de clareza, a notação ϵ indica produções vazias (%empty) e \$ representa o fim do arquivo (EOF).

5.1.1 Tabela Completa de Conjuntos Sintáticos

Tabela 5.1 – Conjuntos FIRST e FOLLOW globais da gramática.

Não-Terminal	FIRST	FOLLOW
program	TYPE, ϵ	\$
global_decl_list	TYPE, ϵ	TYPE, \$
global_decl	TYPE	TYPE, \$
func_decl	TYPE	TYPE, \$
opt_param_list	TYPE, ϵ	PUNCT_CLOSE_PAREN
param_list	TYPE	PUNCT_COMMA, PUNCT_CLOSE_PAREN
stmt_list	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, ϵ	PUNCT_CLOSE_BRACE

Tabela 5.1 – Conjuntos FIRST e FOLLOW globais da gramática.

Não-Terminal	FIRST	FOLLOW
stmt	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
var_decl	TYPE	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE, \$
id_list	ID	PUNCT_SEMICOLON
id_decl	ID	PUNCT_COMMA, PUNCT_SEMICOLON
assign_stmt	ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
if_stmt	IF	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE

Tabela 5.1 – Conjuntos FIRST e FOLLOW globais da gramática.

Não-Terminal	FIRST	FOLLOW
else_clause	ELSE, ϵ	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
while_stmt	WHILE	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
print_stmt	PRINT	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
print_list	ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN	PUNCT_COMMA, PUNCT_CLOSE_PAREN
read_stmt	READ	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
read_list	ID	PUNCT_COMMA, PUNCT_CLOSE_PAREN

Tabela 5.1 – Conjuntos FIRST e FOLLOW globais da gramática.

Não-Terminal	FIRST	FOLLOW
return_stmt	RETURN	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE
block	PUNCT_OPEN_BRACE	TYPE, ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, IF, WHILE, PRINT, READ, RETURN, PUNCT_OPEN_BRACE, PUNCT_CLOSE_BRACE, ELSE, TYPE, \$
opt_expr	ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, ϵ	PUNCT_SEMICOLON
func_call	ID	PUNCT_CLOSE_PAREN, PUNCT_COMMA, PUNCT_SEMICOLON, OR, AND, EQOP, RELOP, PLUS, MINUS, MULT, DIV, POW
opt_func_call_list	ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN, ϵ	PUNCT_CLOSE_PAREN

Tabela 5.1 – Conjuntos FIRST e FOLLOW globais da gramática.

Não-Terminal	FIRST	FOLLOW
func_call_list	ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN	PUNCT_COMMA, PUNCT_CLOSE_PAREN
primary_expr	ID, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN	PUNCT_CLOSE_PAREN, PUNCT_COMMA, PUNCT_SEMICOLON, OR, AND, EQOP, RELOP, PLUS, MINUS, MULT, DIV, POW
literal	INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL	PUNCT_CLOSE_PAREN, PUNCT_COMMA, PUNCT_SEMICOLON, OR, AND, EQOP, RELOP, PLUS, MINUS, MULT, DIV, POW
expr	ID, MINUS, NOT, INTEGER_LITERAL, FLOAT_LITERAL, STR_LITERAL, BOOL_LITERAL, PUNCT_OPEN_PAREN	PUNCT_CLOSE_PAREN, PUNCT_COMMA, PUNCT_SEMICOLON, OR, AND, EQOP, RELOP, PLUS, MINUS, MULT, DIV, POW

6 ESTADOS DO AUTÔMATO LR(0) COM CONFLITOS

A análise do arquivo `parser.output` (gerado com a flag `-report=all` na execução do comando de compilação do *Bison*) revela que o *parser* identificou e solucionou, por meio das configurações de precedência, conflitos esperados de *shift/reduce* ocasionados pela derivação de expressões matemáticas e lógicas. Os estados do autômato LR(0) que atestam esses conflitos resolvidos são **32, 33, 53, 54, 55, 56, 57, 58, 59 e 60**. Em particular para o escopo do operador de menos (`-`), o **Estado 33** trata a ambiguidade do menos unário (`UMINUS`), assegurando que a derivação `MINUS expr` aplique a redução por possuir precedência mais alta em relação às demais resoluções binárias no fluxo. Por outro lado, os estados 53-60 se encarregam das ambiguidades estruturais em operações como precedência de adições, subtrações, relações binárias e lógicas, todas mitigadas pelas declarações de `%left`. É importante notar que o recorrente e famoso obstáculo gramatical do *dangling else* sugerido **não se manifesta em nenhum conflito no autômato deste projeto**. Tal fato decorre pois a regra elaborada para desvio condicional (`if_stmt`) impõe obrigatoriamente o aninhamento sintático por chaves através do não-terminal `block`, neutralizando a ambiguidade da cláusula `ELSE` vista no *dangling else* e provando a robustez do parser.

7 ARQUIVO DE TESTE E SAÍDA

A validação global do analisador sintático foi realizada utilizando um programa fonte estruturado para cobrir as produções e fluxos da linguagem.

7.1 Código de Entrada (entrada.txt)

```
int g_var = 42, g_vazio;

int sem_parametro() {
    return;
}

float test_completo(int x, boolean b) {
    float y = 2.5;
    string s = "Resultado: ";

    read(x);

    if (x > 0 && !b) {
        y = x * (y + 1.0) / 2.0;
        print(s, y);
    } else if (x == 0) {
        sem_parametro();
        return -1.0;
    } else {
        while (b || x < -5) {
            x = x + 1;
        }
    }

    return y;
}
```

7.2 Saída Gerada pelo Parser

Abaixo encontra-se o log completo de deslocamentos e reduções gerado durante a execução da ferramenta sobre o arquivo de teste:

```

1(1): int (TYPE (TYPE_INT))
1(5): g_var (ID)
1(11): = (ASSIGN_OP)
1(13): 42 (INTEGER_LITERAL)
1(15): , (PUNCT_COMMA)
1(17): g_vazio (ID)
1(24): ; (PUNCT_SEMICOLON)
3(1): int (TYPE (TYPE_INT))
3(5): sem_parametro (ID)
3(18): ( (PUNCT_OPEN_PAREN)
3(19): ) (PUNCT_CLOSE_PAREN)
3(21): { (PUNCT_OPEN_BRACE)
4(5): return (RETURN)
4(11): ; (PUNCT_SEMICOLON)
5(1): } (PUNCT_CLOSE_BRACE)
7(1): float (TYPE (TYPE_FLOAT))
7(7): test_completo (ID)
7(20): ( (PUNCT_OPEN_PAREN)
7(21): int (TYPE (TYPE_INT))
7(25): x (ID)
7(26): , (PUNCT_COMMA)
7(28): boolean (TYPE (TYPE_BOOL))
7(36): b (ID)
7(37): ) (PUNCT_CLOSE_PAREN)
7(39): { (PUNCT_OPEN_BRACE)
8(5): float (TYPE (TYPE_FLOAT))
8(11): y (ID)
8(13): = (ASSIGN_OP)
8(15): 2.5 (FLOAT_LITERAL)
8(18): ; (PUNCT_SEMICOLON)
9(5): string (TYPE (TYPE_STR))

```

```

9(12): s (ID)
9(14): = (ASSIGN_OP)
9(16): "Resultado: " (STR_LITERAL)
9(29): ; (PUNCT_SEMICOLON)
11(5): read (READ)
11(9): ( (PUNCT_OPEN_PAREN)
11(10): x (ID)
11(11): ) (PUNCT_CLOSE_PAREN)
11(12): ; (PUNCT_SEMICOLON)
13(5): if (IF)
13(8): ( (PUNCT_OPEN_PAREN)
13(9): x (ID)
13(11): > (RELOP (RELOP_GT))
13(13): 0 (INTEGER_LITERAL)
13(15): && (AND)
13(18): ! (LEXICAL_ERROR)
13(19): b (ID)
13(20): ) (PUNCT_CLOSE_PAREN)
13(22): { (PUNCT_OPEN_BRACE)
14(9): y (ID)
14(11): = (ASSIGN_OP)
14(13): x (ID)
14(15): * (ARITH_OP_MUL)
14(17): ( (PUNCT_OPEN_PAREN)
14(18): y (ID)
14(20): + (ARITH_OP_ADD)
14(22): 1.0 (FLOAT_LITERAL)
14(25): ) (PUNCT_CLOSE_PAREN)
14(27): / (ARITH_OP_DIV)
14(29): 2.0 (FLOAT_LITERAL)
14(32): ; (PUNCT_SEMICOLON)
15(9): print (PRINT)
15(14): ( (PUNCT_OPEN_PAREN)
15(15): s (ID)

```

```

15(16): , (PUNCT_COMMA)
15(18): y (ID)
15(19): ) (PUNCT_CLOSE_PAREN)
15(20): ; (PUNCT_SEMICOLON)
16(5): } (PUNCT_CLOSE_BRACE)
16(7): else (ELSE)
16(12): if (IF)
16(15): ( (PUNCT_OPEN_PAREN)
16(16): x (ID)
16(18): == (EQOP (EQOP_EQ))
16(21): 0 (INTEGER_LITERAL)
16(22): ) (PUNCT_CLOSE_PAREN)
16(24): { (PUNCT_OPEN_BRACE)
17(9): sem_parametro (ID)
17(22): ( (PUNCT_OPEN_PAREN)
17(23): ) (PUNCT_CLOSE_PAREN)
17(24): ; (PUNCT_SEMICOLON)
18(9): return (RETURN)
18(16): - (ARITH_OP_SUB)
18(17): 1.0 (FLOAT_LITERAL)
18(20): ; (PUNCT_SEMICOLON)
19(5): } (PUNCT_CLOSE_BRACE)
19(7): else (ELSE)
19(12): { (PUNCT_OPEN_BRACE)
20(9): while (WHILE)
20(15): ( (PUNCT_OPEN_PAREN)
20(16): b (ID)
20(18): || (OR)
20(21): x (ID)
20(23): < (RELOP (RELOP_LT))
20(25): - (ARITH_OP_SUB)
20(26): 5 (INTEGER_LITERAL)
20(27): ) (PUNCT_CLOSE_PAREN)
20(29): { (PUNCT_OPEN_BRACE)

```

```
21(13): x (ID)
21(15): = (ASSIGN_OP)
21(17): x (ID)
21(19): + (ARITH_OP_ADD)
21(21): 1 (INTEGER_LITERAL)
21(22): ; (PUNCT_SEMICOLON)
22(9): } (PUNCT_CLOSE_BRACE)
23(5): } (PUNCT_CLOSE_BRACE)
25(5): return (RETURN)
25(12): y (ID)
25(13): ; (PUNCT_SEMICOLON)
26(1): } (PUNCT_CLOSE_BRACE)
```

7.3 Tabela de Símbolos Resultante

Após a conclusão com sucesso do *parsing*, os seguintes identificadores foram armazenados e gerenciados na tabela de símbolos:

Tabela 7.1 – Conteúdo final da tabela de símbolos para o arquivo de teste.

Posição	Token	Lexema
3	ID	sem_parametro
2	ID	g_vazio
4	ID	test_completo
6	ID	b
8	ID	s
5	ID	x
7	ID	y
1	ID	g_var

8 CONCLUSÃO

O desenvolvimento do analisador sintático atingiu os objetivos propostos para esta etapa. A integração com o analisador léxico desenvolvido anteriormente foi consolidada com sucesso, e a gramática definida é expressiva para descrever as principais construções da linguagem, sem conflitos irresolvidos no autômato gerado pelo *Bison* (arquivo *parser.output*).

As decisões de projeto adotadas contribuíram para uma gramática completa e de fácil manutenção. Os desafios encontrados durante a implementação (em especial a integração *Flex-Bison* e a modelagem da gramática) foram superados e serviram como aprendizado prático sobre as ferramentas e detalhes dos *parsers*.

O arquivo de teste validou os principais fluxos da linguagem, confirmando o funcionamento correto do parser para as construções implementadas. Assim, o projeto encontra-se pronto para servir de base à próxima etapa do compilador.